



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παραλληλα Συστήματα
Εργασία 1

Ονοματεπώνυμο: ΠΑΚΙΤΣΑΚ ΝΤΕΛΙΑ-ΜΑΡΙΑ /ΙΑΣΩΝ ΚΑΡΑΠΡΟΔΡΟΜΙΔΗΣ
Αριθμός Μητρώου: 1115202200125 / 1115202200064

Περιεχόμενα

Εισαγωγή	2
1 Ασκήση 1	2
1.1 Περιγραφή Προβλήματος	2
1.2 Περιγραφή Λύσης	2
1.3 Πειραματικά αποτελέσματα και σχολιασμός	2
2 Ασκήση 2	4
2.1 Περιγραφή Προβλήματος	4
2.2 Περιγραφή Λύσης	4
2.3 Πειραματικά αποτελέσματα και σχολιασμός	4
3 Ασκήση 3	6
3.1 Περιγραφή Προβλήματος	6
3.2 Περιγραφή Λύσης	6
3.3 Πειραματικά Αποτελέσματα και Σχολιασμός	6
4 Ασκήση 4	10
4.1 Περιγραφή Προβλήματος	10
4.2 Περιγραφή Λύσης	10
4.3 Πειραματικά αποτελέσματα και σχολιασμός	10
5 Ασκήση 5	14
5.1 Περιγραφή Προβλήματος	14
5.2 Περιγραφή Λύσης	14
5.3 Πειραματικά αποτελέσματα και σχολιασμός	15
6 Όλα τα γραφήματα της ασκήσης 4	16

Εισαγωγή

Για την παραγωγή γραφημάτων χρησιμοποιήσαμε *script* σε *python*. Όλες οι σκησεις εκτελεστήκαν σε AMD Ryzen 7 5700X 8 cores 16 threads 16 gb ram. Οι χρόνοι στα πειράματα είναι ο μέσος όρος από 4-5 εκτελέσεις

1 Ασκήση 1

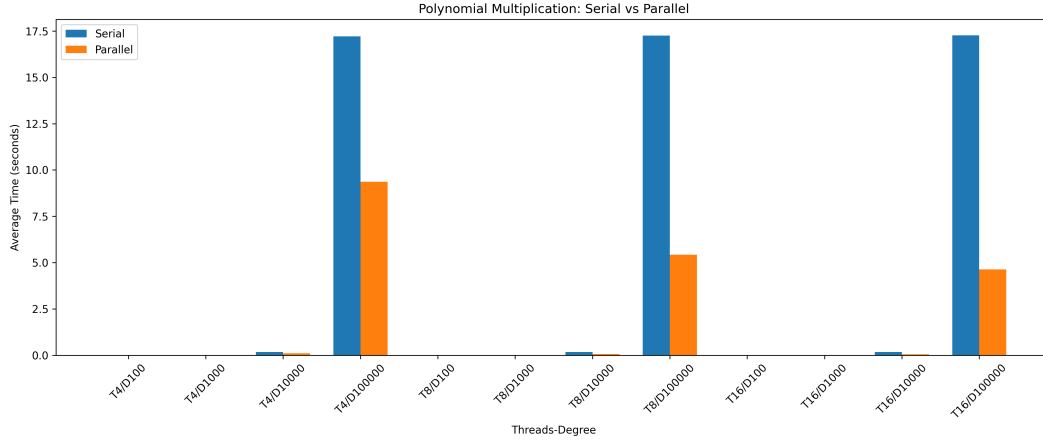
1.1 Περιγραφή Προβλήματος

Η ασκήση ζητά να υλοποιήσουμε ένα πρόγραμμα που υπολογίζει τον πολλαπλασιασμό δυο τυχαίων πλήρων πολυωνυμικών βαθμού n με μη μηδενικούς ακέραιους συντελεστές τόσο με σειριακό όσο και με παραλληλό αλγόριθμο χρησιμοποιώντας Πηρεαδς. Στην συνέχεια θα πρέπει να μετρήσουμε και να συγκρίνουμε τους χρόνους εκτέλεσης για την αρχικοποίηση του προβλήματος, του σειριακού υπολογισμού και του παραλλήλου υπολογισμού. Τέλος θα πρέπει να επιβεβαιώσουμε ότι τα αποτελέσματα των δυο αλγορίθμων (σειριακό/παραλληλό) είναι ίδια και να μελετήσουμε την απόδοση για διαφορετικό αριθμό νημάτων και διαφορετικό αριθμό βαθμού πολυωνυμικών.

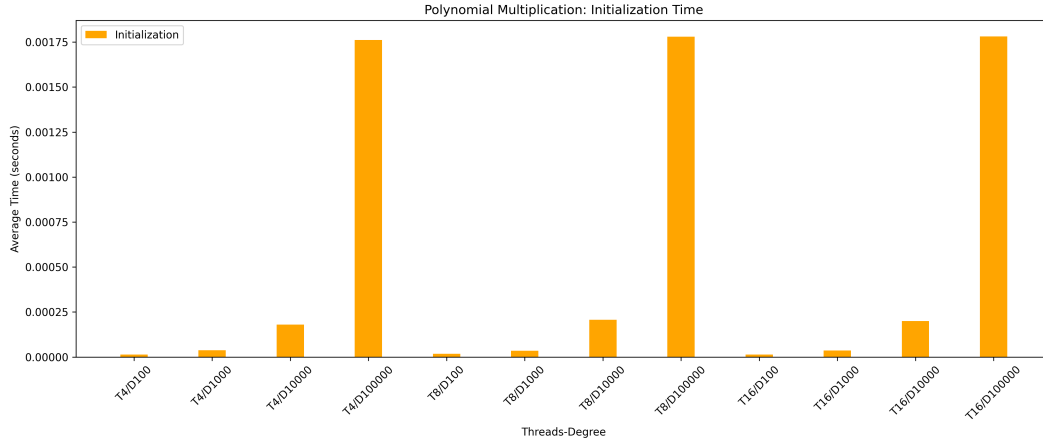
1.2 Περιγραφή Λύσης

Αρχικά το πρόγραμμα ελέγχει αν τα ορίσματα που δέχεται είναι σωστά, δηλαδή αν δόθηκαν τιμές για τον βαθμό των πολυωνυμικών και τον αριθμό των *thread*. Δημιουργούμε δυναμικά τους πίνακες που θα περιέχουν τους τυχαίους μη μηδενικούς συντελεστές. Στην συνέχεια αφού δημιουργήσουμε τους τυχαίους συντελεστές με την χρήση της συναρτήσης *generate_polynomials* και μετρήσουμε τον χρόνο αρχικοποίησης θα εκτελέσουμε τον σειριακό πολλαπλασιασμό με διπλή επανάληψη. Ο κάθε συντελεστής του αποτελέσματος προκύπτει από το άθροισμα γινόμενων των συντελεστών των δυο πίνακων που περιέχουν τους συντελεστές των πολυωνυμικών. Αντιστοίχα για τον παραλληλό αλγόριθμο δημιουργούμε νήματα καθένα από τα οποία θα επεξεργάζονται ένα συγκεκριμένο τμήμα υπολογίζοντας τις αντίστοιχες τιμές χωρίς να απαιτείται κάποιος συγχρονισμός καθώς τα τμήματα είναι ανεξάρτητα και δεν επηρεάζονται αφού έχουν ξεχωριστό ευρος δεικτών. Μετά την ολοκλήρωση των αλγορίθμων συγκρίνουμε τα αποτελέσματα του σειριακού αλγορίθμου και του παραλλήλου αλγορίθμου για επιβεβαίωση ορθότητας.

1.3 Πειραματικά αποτελέσματα και σχολιασμός



(α') Serial vs Parallel



(β') Execution Initialization time

Summary of Average Times and Speedup

Threads-Degree	Serial	Parallel	Improvement (Serial/Parallel)
T4/D100	0.000025	0.000248	0.10x
T4/D1000	0.002065	0.001351	1.53x
T4/D10000	0.172261	0.096629	1.78x
T4/D100000	17.208906	9.357942	1.84x
T8/D100	0.000027	0.000368	0.07x
T8/D1000	0.002085	0.000805	2.59x
T8/D10000	0.172776	0.055352	3.12x
T8/D100000	17.263354	5.425441	3.18x
T16/D100	0.000024	0.000564	0.04x
T16/D1000	0.002101	0.000916	2.29x
T16/D10000	0.173656	0.048023	3.62x
T16/D100000	17.268774	4.624229	3.73x

(γ') Execution time table

$\Sigma\chi\eta\mu$ 1: Polynomial multiplication time.

Όπως παρατηρούμε από τα πειραματικά αποτελέσματα που συλλεχθήκαν από το *script.py* η απόδοση του παραλλήλου αλγορίθμου εξαρτάται έντονα από τον βαθμό του πολυωνυμίου και τον αριθμό των *threads*. Για μικρούς βαθμούς (π.χ. 100) ο σειριακός αλγόριθμος είναι ταχύτερος καθώς το κόστος δημιουργίας και συγχρονισμού των νημάτων υπερβαίνει το χρόνο που κερδίζουμε από την παραλληλή εκτέλεση. Πιο συγκεκριμένα όταν φτίνουμε πολλά *threads* το λειτουργικό σύστημα πρέπει να τα δημιουργήσει και να τα συγχρονίσει. Αυτός ο χρόνος δεν συλλαμβάνεται άμεσα στον υπολογισμό αλλά είναι απαραίτητος για να λειτουργήσει η παραλληλία. Αντιθέτως, για μεγάλους βαθμούς (π.χ. 10^5) η παραλληλή εκτέλεση είναι πιο αποδοτική που φτάνει έως και $1.75x$ σε σχέση με τον σειριακό, όπως φαίνεται στον πίνακα. Αυτό συμβαίνει επειδή η υπολογιστική πολυπλοκότητα αυξάνεται εκθετικά με την αύξηση του βαθμού του πολυωνυμίου.

Η επιτάχυνση που παρατηρείται οφείλεται στο γεγονός ότι ο αλγόριθμος επιτρέπει σε κάθε *thread* να επεξεργάζεται ανεξάρτητο τμήμα του πίνακα αποτελεσμάτων αποφεύγοντας συγχρούσεις και αναγκάζει για συγχρονισμό. Ο μόνος συγχρονισμός που χρησιμοποιούμε είναι η χρήση της *pthread_join* για να περιμένουμε την ολοκλήρωση όλων των νημάτων. Δεν χρησιμοποιήθηκαν *mutex* ή *rwlock* για την προστασία κάποιων κοινών μεταβλητών αφού δεν υπήρχε κοινή πρόσβαση σε ίδιες θέσεις μνήμης.

Συμπερασματικά η παραλληλή υλοποίηση του αλγορίθμου πολλαπλασιασμού πολυωνύμων παρουσιάζει σημαντική επιτάχυνση για μεγάλους βαθμούς και αυξημένο αριθμό νημάτων. Η απόδοση βελτιώνεται καθώς αυξάνεται το μέγεθος του προβλήματος. Η πολυπλοκότητα του αλγορίθμου είναι $O(n^2)$ καθώς για κάθε συντελεστή του πρώτου πολυωνυμίου γίνονται πολλαπλασιασμοί με όλους τους συντελεστές του δεύτερου. Για παράδειγμα αν έχουμε βαθμό 10^5 ο συνολικός αριθμός πράξεων είναι 10^{10} . Ένας σειριακός αλγόριθμος πρέπει να εκτελέσει όλες αυτές τις πράξεις διαδοχικά, ενώ στην παραλληλή εκτέλεση οι πράξεις μοιράζονται σε πολλά *threads* μειώνοντας τον πραγματικό χρόνο εκτέλεσης.

2 Ασκήση 2

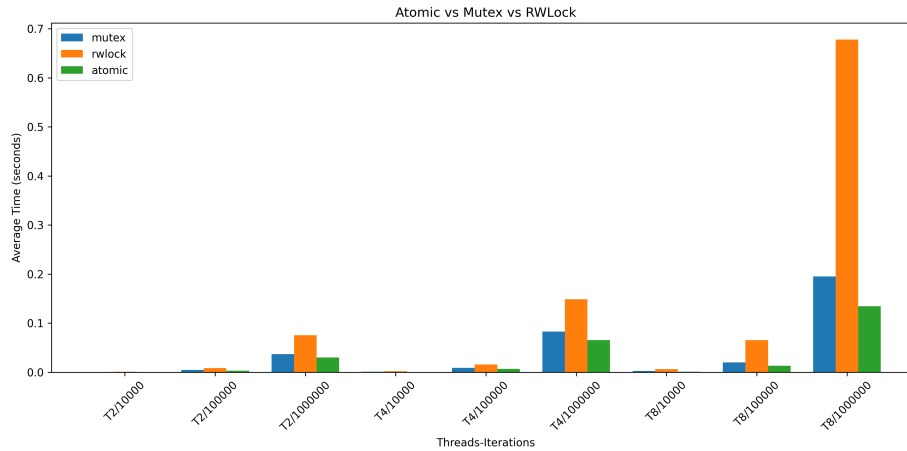
2.1 Περιγραφή Προβλήματος

Συμφώνα με την εκφώνηση η ασκήση μας ζητάει να ενημερώσουμε μια κοινοχρήστη μεταβλητή που αρχικοποιείται με 0. Κάθε νήμα θα εκτελεί έναν σταθερό αριθμό επαναλήψεων και σε κάθε βήμα θα αυξούμε κατά ένα αυτή την κοινοχρήστη μεταβλητή. Επειδή η ταυτόχρονη πρόσβαση θα δημιουργήσει *race conditions* η ασκήση απαιτεί την υλοποίηση τριών διαφορετικών τρόπων συγχρονισμού για να διασφαχιστεί η ορθότητα της ασκήσης. Πρώτος τρόπος είναι με χρήση *mutex locks*, ο δεύτερος με χρήση *read – write locks* και ο τρίτος με χρήση *atomic* εντολών. Στην συνέχεια θα πρέπει να συγκρίνουμε την επίδοση των τριών τρόπων για διαφορετικό αριθμό επαναλήψεων και νημάτων.

2.2 Περιγραφή Λύσης

Το πρόγραμμα δέχεται 3 ορίσματα, το *mode(mutex/rwlock/atomic)*, τον αριθμό των *thread* και τον αριθμό των επαναλήψεων. Η μεταβλητή *mode* προστέθηκε για να αποφύγω την δημιουργία 3 διαφορετικών αρχείων. Στην συνέχεια δημιουργούμε και αρχικοποιούμε όλες τις απαραίτητες δομές για τον συγχρονισμό, δηλαδή το *mutex* και *rwlock* ανάλογα με την επιλογή του χρήστη ενώ για την περίπτωση του *atomic* δεν απαιτείται κάποια επιπλέον δομή. Η *main* θα είναι υπεύθυνη για την δημιουργία των νημάτων καθώς και την αναμονή της ολοκλήρωσης τους *pthread_join*. Σε κάθε *thread* θα περνάμε μια δομή *thread_info* που περιέχει το *id* και τον αριθμό των επαναλήψεων. Η συνάρτηση *thread_function* αυξάνει κατά ένα την κοινοχρήστη μεταβλητή με τον κατάλληλο μηχανισμό συγχρονισμού (*mutex/rwlock/atomic*). Με αυτόν τον τρόπο αποφεύγουμε τα *race conditions*. Τέλος αποδεσμεύουμε από την μνήμη την δομή και τα *locks* που δεσμεύσαμε κατά την διάρκεια του προγράμματος.

2.3 Πειραματικά αποτελέσματα και σχολιασμός



(α') Execution time graph

Summary of Average Times (Atomic vs Mutex vs RWLock)

Threads-Iterations	mutex	rlock	atomic
T2/10000	0.000525	0.000958	0.000267
T2/100000	0.004291	0.008021	0.003212
T2/1000000	0.036817	0.075387	0.029822
T4/10000	0.000905	0.001781	0.000328
T4/100000	0.008524	0.015749	0.006480
T4/1000000	0.082434	0.148811	0.065220
T8/10000	0.002268	0.006314	0.000968
T8/100000	0.019658	0.065443	0.013080
T8/1000000	0.195052	0.677976	0.134552

(β') Execution time table

Σχήμα 2: Execution time.

Όπως βλέπουμε παραπάνω βάσει των πειραματικών αποτελεσμάτων που συλλεχθήκαν από το *script.py* παρατηρούμε κάποιες βασικές διαφορές χρόνων μεταξύ των τριών μηχανισμών συγχρονισμού. Ξεκινάμε παρατηρώντας ότι για κάθε αριθμό νημάτων/αριθμό επαναλήψεων η *atomic* προσεγγίση είναι η ταχύτερη στην συνέχεια ακολουθεί το *mutex* και τέλος το *rlock*. Η διαφορά στην επιδοση ανάμεσα στις προσεγγίσεις εξαρτάται από τον τρόπο που υλοποιούνται. Αρχικά οι ατομικές εντολές εκτελούνται απευθείας από την *cpu*, δηλαδή δεν απαιτούν κλειδώμα ούτε εισόδο στον *kernel*. Το *mutex* για κάθε *thread* που θέλει να ενημερώσει τη μεταβλητή αποκτά ένα *lock* κάνει την ενημέρωση (αύξηση) και απελευθερώνει το *lock*. Τέλος το *readwrite lock* επιτρέπει σε πολλά νήματα να αποκτήσουν ταυτόχρονα τις πληροφορίες (*read – lock*) χωρίς να εμποδίζουν ένα το άλλο, όμως μόνο ένα *thread* μπορεί να αποκτήσει *write – lock* και αποκλείει όλους τους υπολοίπους αναγνώστες και εγγραφείς. Αυτός ο τρόπος συγχρονισμού είναι πιο αποδοτικός για σενάρια όπου έχουμε πολλές αναγνώσεις και λίγες εγγραφές. Στην περίπτωση μας όμως όλα τα νήματα κάνουν μόνο εγγραφές και άρα το *rlock* λειτουργεί πρακτικά σαν *mutex* αλλά με μεγαλύτερο κόστος καθώς η υλοποίηση είναι πιο συνθετική.

Συμφώνα με τους χρόνους των σχημάτων παραπάνω παρατηρούμε ότι καθώς αυξάνονται οι επαναλήψεις με σταθερό αριθμό των *thread* αυξάνεται ο χρόνος για κάθε προσεγγίση. Για παράδειγμα για 4 *threads* παρατηρούμε ότι για 10.000 επαναλήψεις ο χρόνος εκτέλεσης με *mutex* 0.000905, για 100.000 είναι 0.008524 και για 1.000.000 είναι 0.082434. Είναι εμφανές ότι όσο αυξάνονται οι επαναλήψεις η διαφορά αυξάνεται στον χρόνο αυξάνεται εξίσου διότι όσο περισσότερες επαναλήψεις ζητάμε τόσο περισσότερες αυξήσεις πρέπει να γίνουν συνολικά. Αντιστοίχα ισχύει και για τις άλλες δύο προσεγγίσεις (*rw.lock*, *atomic*) όπως παρατηρούμε στον πίνακα.

Όσο αυξάνεται ο αριθμός νημάτων με σταθερό αριθμό επαναλήψεων παρατηρούμε και εδώ την αύξηση στον χρόνο. Για παράδειγμα με σταθερό αριθμό επαναλήψεων 10^6 για την *mutex* προσεγγίση και αριθμό νημάτων

2 έχουμε 0.036817, για αριθμο νημάτων 4 έχουμε 0.082434 και για αριθμο νημάτων 8 έχουμε 0.195052. Αυτό συμβαίνει διότι όλα τα νήματα προσπαθούν να ενημερώσουν την ίδια κοινοχρηστή μεταβλητή και οσα περισσότερα νήματα τόσο πιο συχνά συγκρούονται οι προσπάθειες προσβάσης και χρειάζεται να περιμένουν το *lock* η να ολοκληρωθεί η *atomic* εντολή.

Συμπερασματικά η *atomic* προσεγγίση είναι πιο αποδοτική για το συγκεκριμένο προσβλημα ειδικα όταν ο αριθμος των επαναληψεων και νημάτων είναι μεγалος .

3 Ασκήση 3

3.1 Περιγραφή Προβλήματος

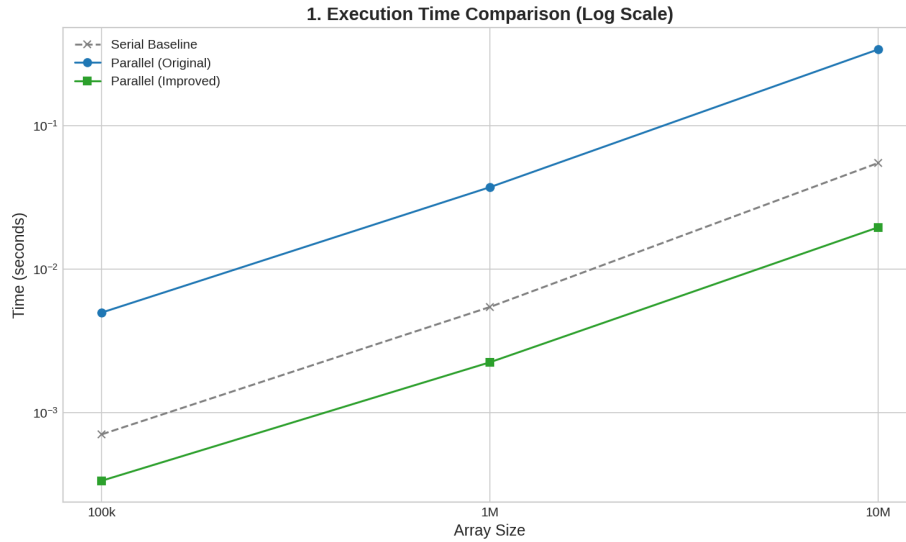
Σε αυτή την άσκηση έπρεπε να υλοποιήσουμε ένα παράλληλο πρόγραμμα στο οποίο το κάθε νήμα να παίρνει έναν από 4 ακεραίους πίνακες N θέσεων και να υπολογίζει πόσα στοιχεία είναι μη μηδενικά στον πίνακα που του έχει ανατεθεί, καθώς και ένα σειριακό πρόγραμμα που κάνει την ίδια δουλειά και στο τέλος να τυπώνει τον χρόνο που χρειάστηκε.

3.2 Περιγραφή Λύσης

Για αρχή, ελέγχω αν τα ορίσματα του προγράμματος είναι σωστά και μετά αρχικοποιώ τις απαραίτητες δομές, μετρώντας τα μη μηδενικά στοιχεία του πίνακα κατά την αρχικοποίηση για έλεγχο ορθότητας των προγραμμάτων στο τέλος. Χρησιμοποιώ τη `my_rand()` από τον κώδικα του βιβλίου, διότι είναι πιο γρήγορη από τη `rand()`, και επίσης χρησιμοποιώ μόνο μία τυχαία τιμή και για τους 4 πίνακες, καθώς δεν χρειάζεται παραπάνω, οπότε γλιτώνω αρκετές κλήσεις της συνάρτησης.

Το παράλληλο πρόγραμμα ελέγχει ποιο είναι το `rank` του και μετρά τα μη μηδενικά στοιχεία του πίνακα στη θέση `rank`, ενημερώνοντας το αντίστοιχο πεδίο στην `array_stats_s`. Δεν χρειάστηκε συγχρονισμός, διότι το κάθε νήμα έχει το δικό του πεδίο και τον δικό του πίνακα, τα οποία δεν επεξεργάζεται κανένα άλλο νήμα, άρα δεν υπάρχει κάποιο `race condition`. Όταν επιστρέψουν όλα τα νήματα, τρέχω το σειριακό πρόγραμμα και μετά ελέγχω την ορθότητα των δύο εκτελέσεων. Τέλος, αποδεσμεύω ό,τι μνήμη δέσμευσα για τις δομές.

3.3 Πειραματικά Αποτελέσματα και Σχολιασμός



(α') Execution time graph

5. Execution Time Summary Table

Array Size	Init Time	Serial Time	Par. Orig.	Par. Imp.
100k	0.00183 s	0.00071 s	0.00498 s	0.00034 s
1M	0.01563 s	0.00545 s	0.03718 s	0.00224 s
10M	0.15327 s	0.05482 s	0.33854 s	0.01953 s

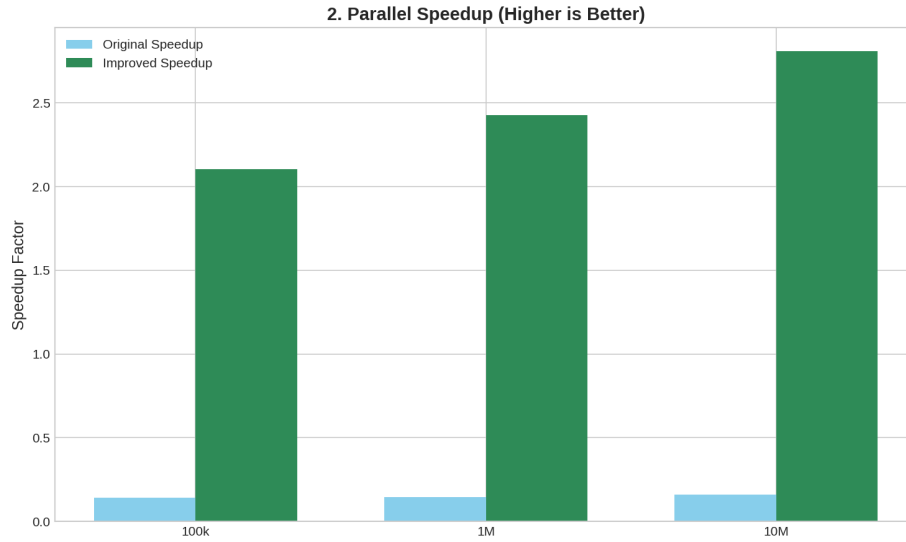
(β') Execution time table

Σχήμα 3: Comparison of execution times and data table.

Όπως φαίνεται και στα γραφήματα, η απόδοση του παράλληλου προγράμματος είναι πολύ χαμηλή. Αυτό συμβαίνει διότι η δομή `array_stats_s` έχει μέγεθος 32 bytes (4 πεδία τύπου `long long int` με μέγεθος 8 bytes). Το μέγεθος του cache line της CPU είναι 64 bytes, άρα η `array_stats_s` χωράει μέσα σε ένα cache line.

Επειδή κάθε νήμα προσπαθεί να προσπελάσει το ίδιο cache line, υπάρχει false sharing, οπότε κάθε φορά που γίνεται κάποια ενημέρωση δημιουργείται cache miss. Ο χρόνος που απαιτείται για να ξαναφορτωθεί το cache line στην μνήμη σας είναι μεγαλύτερος από τον χρόνο που κερδίζουμε με την παράλληλη εκτέλεση.

Υπάρχει όμως λύση για αυτό το πρόβλημα: βάζοντας padding ανάμεσα στα πεδία εξαλείφεται το false sharing. Τώρα το κάθε πεδίο καταλαμβάνει 64 bytes, άρα μπαίνει σε διαφορετικό cache line και δεν υπάρχει πλέον σύγκρουση. Η βελτίωση στον χρόνο είναι εμφανής, καθώς το speedup από 0.15 (δηλαδή πιο αργό από το serial) ανέβηκε σε 2.6 για πίνακες μεγέθους 10^6 και σε 2.1 για πίνακες μεγέθους 10^5 .



(α') Speedup graph

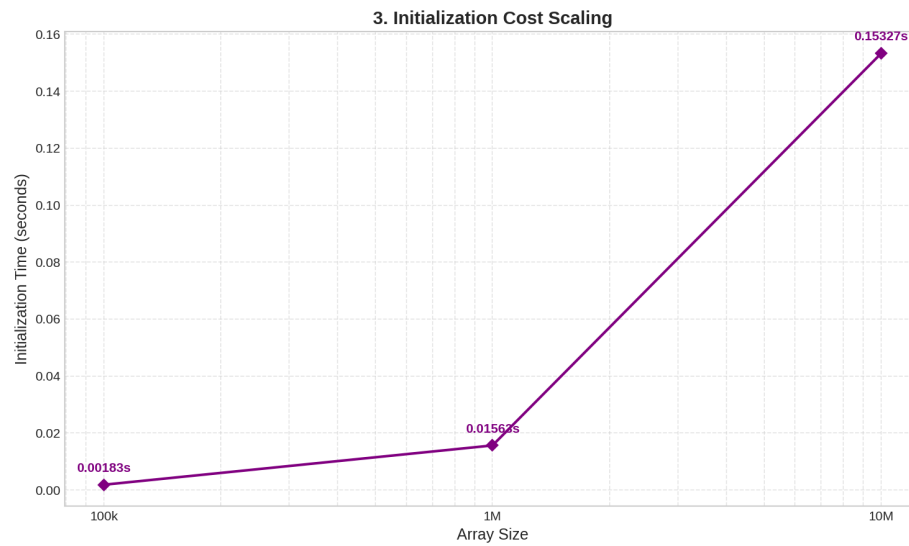
6. Parallel Speedup Factor Table

Array Size	Original Speedup	Improved Speedup
100k	0.14x	2.10x
1M	0.15x	2.43x
10M	0.16x	2.81x

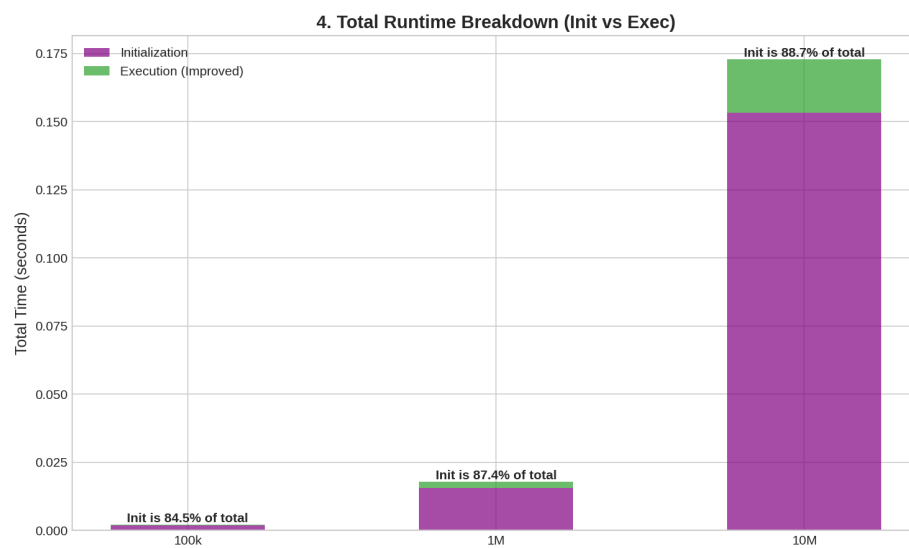
(β') Speedup table

Σχήμα 4: Comparison of execution times and data table.

Τέλος, παρατηρείται ότι το *initialization* αποτελεί το βασικό *bottleneck*, καταναλώνοντας το 80-90% του συνολικού χρόνου. Αυτό οφείλεται στη σειριακή φύση της αρχικοποίησης σε συνδυασμό με τον υψηλό υπολογιστικό φόρτο της. Σύμφωνα με τον νόμο του Amdahl, η μέγιστη δυνατή επιτάχυνση περιορίζεται δραστικά όταν ένα τόσο μεγάλο ποσοστό του προγράμματος παραμένει σειριακό. Επομένως, οποιαδήποτε περαιτέρω βελτιστοποίηση στο παράλληλο τμήμα θα έχει φθίνουσα απόδοση (*diminishing returns*), αν δεν παραλληλοποιηθεί και η αρχικοποίηση, κάτι που ωστόσο δεν απαιτείται στην παρούσα εργασία.



(α') Initialization time graph



(β') overhead analysis

Σχήμα 5: Initialization overhead.

4 Άσκηση 4

4.1 Περιγραφή Προβλήματος

Απαιτήθηκε η προσομοίωση ενός τραπεζικού συστήματος που υποστηρίζει δύο λειτουργίες: τη μεταφορά χρημάτων και την ερώτηση υπολοίπου. Η επιλογή μεταξύ των δύο τύπων συναλλαγών, καθώς και η επιλογή των λογαριασμών στους οποίους αυτές διενεργούνται, πραγματοποιείται με τυχαίο τρόπο.

4.2 Περιγραφή Λύσης

Αρχικά, πραγματοποιείται επεξεργασία και επικύρωση των δεδομένων εισόδου, επιστρέφοντας σφάλμα σε περίπτωση μη έγκυρων τιμών. Στη συνέχεια, αρχικοποιούνται ο πίνακας λογαριασμών και ο πίνακας των thread IDs, ενώ ξεκινά η εκτέλεση του αριθμού των threads που ορίστηκε. Δεδομένου ότι τα νήματα ενδέχεται να εγγράψουν σε οποιονδήποτε λογαριασμό, απαιτείται μηχανισμός συγχρονισμού για τη διασφάλιση της ακεραιότητας των δεδομένων. Τα δύο σχήματα συγχρονισμού που υλοποιήθηκαν είναι τα εξής: Read-Write lock και Mutex.

Επιπλέον, το granularity των κλειδωμάτων, καθώς και το πλήθος των συναλλαγών ανά νήμα, καθορίζονται από τις παραμέτρους εισόδου. Στην προσέγγιση fine-grained, κατά τη μεταφορά χρημάτων απαιτείται η δέσμευση δύο κλειδωμάτων από ένα thread. Παρατηρήθηκε ότι αυτό οδηγούσε συχνά σε deadlock λόγω της ύπαρξης circular wait. Για την αντιμετώπιση του προβλήματος, επιβλήθηκε συγκεκριμένη σειρά στη διαδικασία δέσμευσης (lock ordering). Συγκεκριμένα, τα νήματα δεσμεύουν τα κλειδιά βάσει αύξουσας σειράς του index τους (από το μικρότερο στο μεγαλύτερο), εξαλείφοντας έτσι το φαινόμενο. Τέλος, πραγματοποιείται αποδέσμευση των δομών μνήμης και τερματισμός της λειτουργίας.

4.3 Πειραματικά αποτελέσματα και σχολιασμός

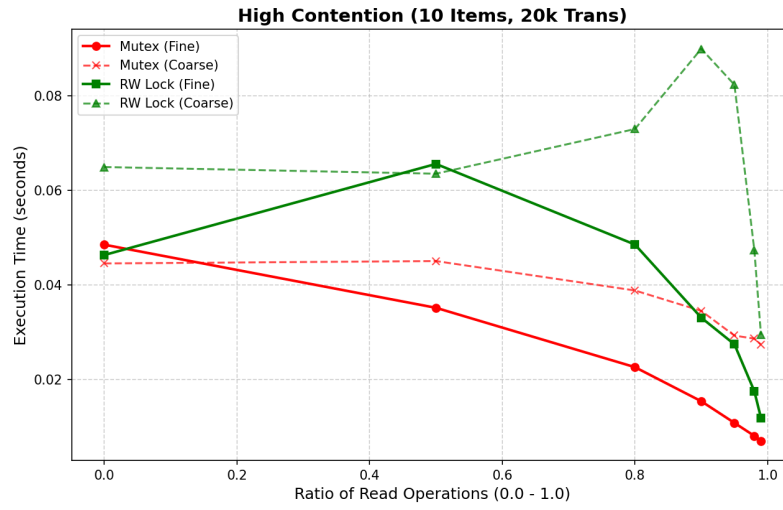
Για την πειραματική διαδικασία επιλέχθηκαν δύο σενάρια: ένα με υψηλό ανταγωνισμό μεταξύ των threads και ένα με μεσαίο, το οποίο θεωρείται και το πλέον αντιπροσωπευτικό σε πραγματικές συνθήκες. Τόσο τα threads όσο και ο αριθμός των συναλλαγών διατηρήθηκαν σταθερά (16 threads και 20.000 συναλλαγές αντίστοιχα). Η απόφαση σταθεροποίησης των συναλλαγών ελήφθη διότι η μεταβολή τους δεν προσέφερε επιπρόσθετη πληροφορία, επιτρέποντας παράλληλα τη διατήρηση της ανάλυσης σε συνοπτικό επίπεδο. Ωστόσο, διενεργήθηκαν πειράματα καλύπτοντας όλο το φάσμα των παραμέτρων και το πλήρες σύνολο γραφημάτων παρατίθεται στο τέλος του παρόντος εγγράφου. Επιπλέον, εξετάστηκαν διακριτά ποσοστά ερωτημάτων υπολοίπου. Τα αποτελέσματα παρουσιάζονται παρακάτω:

Read %	Mut Fine (s)	Mut Crse (s)	RW Fine (s)	RW Crse (s)
0%	0.0485	0.0445	0.0463	0.0649
50%	0.0351	0.0450	0.0656	0.0635
80%	0.0226	0.0388	0.0485	0.0729
90%	0.0154	0.0345	0.0330	0.0899
95%	0.0108	0.0293	0.0274	0.0823
98%	0.0080	0.0286	0.0175	0.0473
99%	0.0070	0.0273	0.0119	0.0294

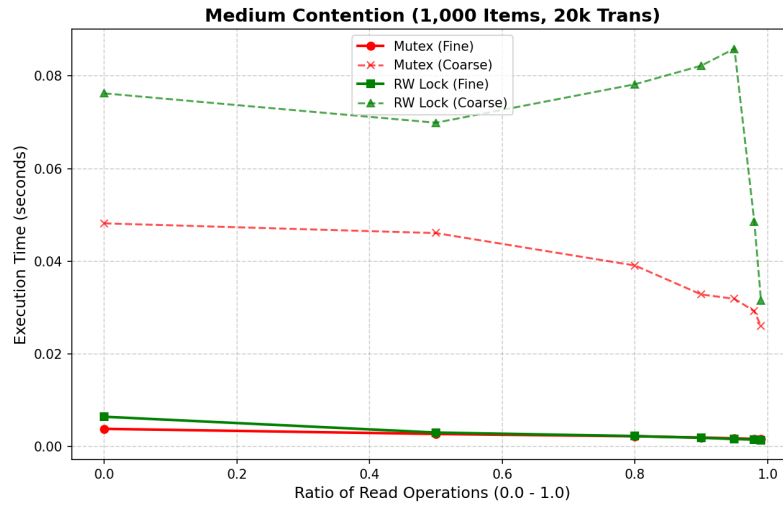
Πίνακας 1: High Contention (10 Items, 20k Trans)

Read %	Mut Fine (s)	Mut Crse (s)	RW Fine (s)	RW Crse (s)
0%	0.0038	0.0482	0.0064	0.0762
50%	0.0027	0.0461	0.0030	0.0699
80%	0.0022	0.0391	0.0022	0.0782
90%	0.0019	0.0328	0.0019	0.0822
95%	0.0018	0.0319	0.0016	0.0859
98%	0.0016	0.0292	0.0015	0.0485
99%	0.0016	0.0260	0.0014	0.0316

Πίνακας 2: Medium Contention (1,000 Items, 20k Trans)



(α') High contention



(β') Medium Contention

Σχήμα 6: Scenario time graphs.

Όπως ήταν αναμενόμενο, η προσέγγιση fine-grained είναι σαφώς ταχύτερη από την coarse-grained. Σε συνθήκες υψηλού ανταγωνισμού, καθίσταται εμφανές το αυξημένο overhead που παρουσιάζουν τα read-write locks έναντι των απλούστερων mutex. Συγκεκριμένα, στα coarse-grained κλειδώματα, τα read-write locks υστερούν σημαντικά σε ταχύτητα σε σύγκριση με τα mutex. Μάλιστα, όταν ο ανταγωνισμός είναι έντονος, ακόμη και τα fine-grained read-write locks αποδεικνύονται πιο αργά από τα coarse-grained mutex.

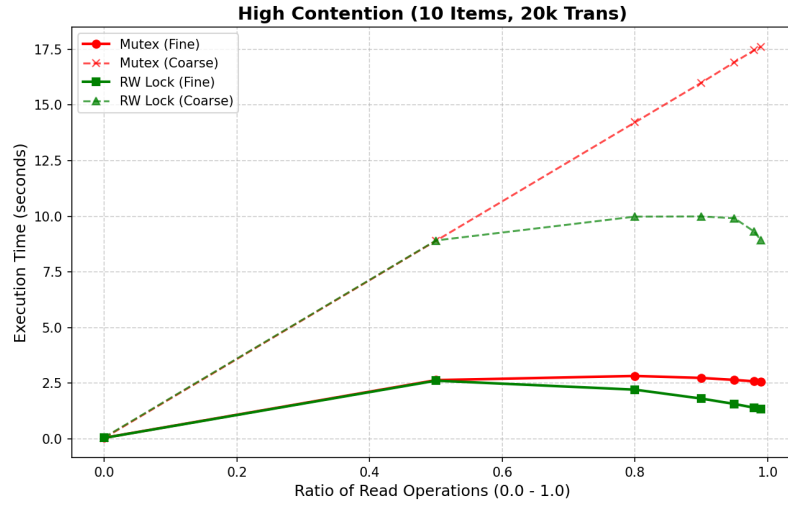
Ωστόσο, καθώς αυξάνεται το ποσοστό των reads της εφαρμογής, το συγκριτικό πλεονέκτημα των mutex περιορίζεται. Όταν δεν υπάρχει υψηλός ανταγωνισμός, το overhead των read-write locks μετριάζεται, με αποτέλεσμα η απόδοσή τους να είναι σχεδόν ίδια με αυτή των mutex. Μέχρι αυτό το σημείο, τα reads εκτελούνταν ταχύτατα, καθώς περιλάμβαναν απλές αριθμητικές πράξεις. Εάν όμως εισάγουμε μια τεχνητή καθυστέρηση μερικών μικροδευτερολέπτων, προσομοιώνοντας μια πιο σύνθετη λειτουργία, η συμπεριφορά του συστήματος διαφοροποιείται ως εξής:

Read %	Mut Fine (s)	Mut Crse (s)	RW Fine (s)	RW Crse (s)
0%	0.0045	0.0501	0.0038	0.0896
50%	0.5541	8.9350	0.6592	8.9383
80%	0.8751	14.2427	1.0403	9.9994
90%	1.0046	16.0051	1.1652	10.0033
95%	1.0414	16.8796	1.2249	9.9412
98%	1.0791	17.4501	1.2608	9.3952
99%	1.0925	17.6241	1.2724	8.7667

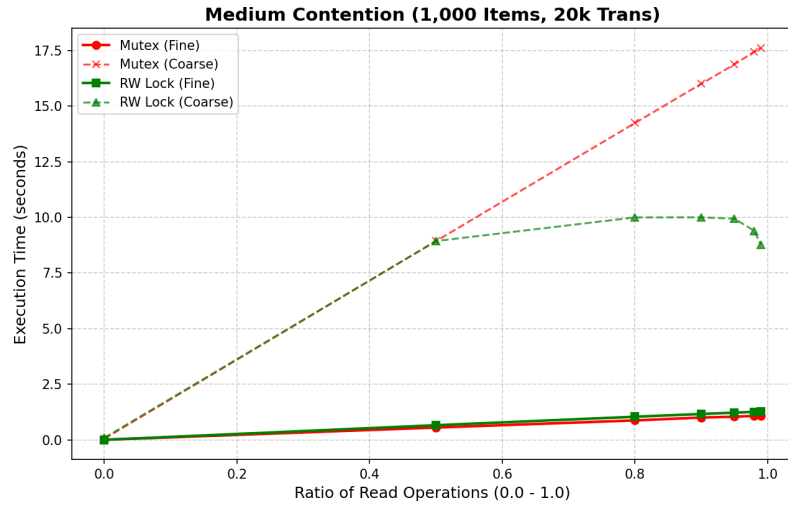
Πίνακας 3: Medium Contention with sleep (1,000 Items, 20k Trans)

Read %	Mut Fine (s)	Mut Crse (s)	RW Fine (s)	RW Crse (s)
0%	0.0511	0.0451	0.0475	0.0729
50%	2.6407	8.8984	2.6176	8.9127
80%	2.8288	14.2157	2.2131	9.9838
90%	2.7400	15.9943	1.8132	9.9910
95%	2.6503	16.9166	1.5706	9.9114
98%	2.5879	17.4579	1.4008	9.3051
99%	2.5714	17.6157	1.3330	8.9161

Πίνακας 4: High Contention (10 Items, 20k Trans)



(α') High contestation



(β') Medium Contestation

Σχήμα 7: Scenario time graphs with sleep.

Όπως προκύπτει από τα αποτελέσματα, το `coarse-grained mutex` παρουσιάζει τη χαμηλότερη απόδοση. Το `coarse-grained read-write lock` είναι σημαντικά ταχύτερο έναντι του `mutex`, καθώς επιτρέπει την ταυτόχρονη ανάγνωση δεδομένων από πολλαπλά νήματα, πλεονέκτημα που καθίσταται εμφανέστερο καθώς αυξάνεται το ποσοστό των `reads`. Όσον αφορά τα `fine-grained κλειδώματα`, σε συνθήκες υψηλού ανταγωνισμού (`high contention`), το `read-write lock` υπερτερεί καθώς επιτρέπει παράλληλες αναγνώσεις στον πίνακα, ενώ όταν ο ανταγωνισμός είναι χαμηλός, οι χρόνοι εκτέλεσης των δύο μεθόδων συγκλίνουν σημαντικά.

5 Ασκήση 5

5.1 Περιγραφή Προβλήματος

Σε αυτή την ασκήση πρέπει να υλοποιήσουμε και να συγκρίνουμε 3 διαφορετικές υλοποιήσεις φραγματος (*barrier*) στον συγχρονισμό νημάτων. Πιο συγκεκριμένα το πρώτο πρόγραμμα χρησιμοποιεί το ετοιμο `pthread_barrier` για να συγχρονίσει νήματα σε κάθε επαναληψη ενός μεγάλου βροχού. Το δεύτερο πρόγραμμά μας ζητάει να υλοποιήσουμε ακριβώς την ίδια διεργασία αλλά να μην χρησιμοποιήσουμε την ετοιμη *barrier* αλλά να την δημιουργήσουμε μόνοι μας με χρήση κλειδώματος (*mutex*) και μεταβλητή συνθήκης (*cond*) χωρίς *busy waiting*. Τέλος στο τρίτο πρόγραμμα θα πρέπει να υλοποιήσουμε την ίδια εργασία με τα προηγούμενα προβλήματα αλλά θα χρησιμοποιεί φράγμα με αναμονή σε εγρηγορήση (*sense – reversal centralized barrier*). Ο στόχος μας είναι να τρεξούμε αυτά τα προγράμματα για διαφορετικό αριθμό νημάτων και επαναληψεων

5.2 Περιγραφή Λύσης

Αρχικά όλα τα προγράμματα θα δέχονται 2 ορίσματα, τον αριθμό νημάτων και τον αριθμό των επαναληψεων.

Στην πρώτη υλοποίηση αξιοποιούμε τον ετοιμο μηχανισμό που μας παρέχεται από την βιβλιοθήκη *threads*. Ορίζουμε ένα κοινό *barrier* το οποίο το αρχικοποιούμε ώστε να περιμένει όλα τα νήματα. Κάθε νήμα εκτελεί ένα βροχο με έναν συγκεκριμένο αριθμό επαναληψεων και σε κάθε επαναληψη καλείται η `pthread_barrier_wait` για να εξασφαλιστεί πως θα προχωρήσουν τα νήματα ταυτόχρονα στο επομένο βήμα.

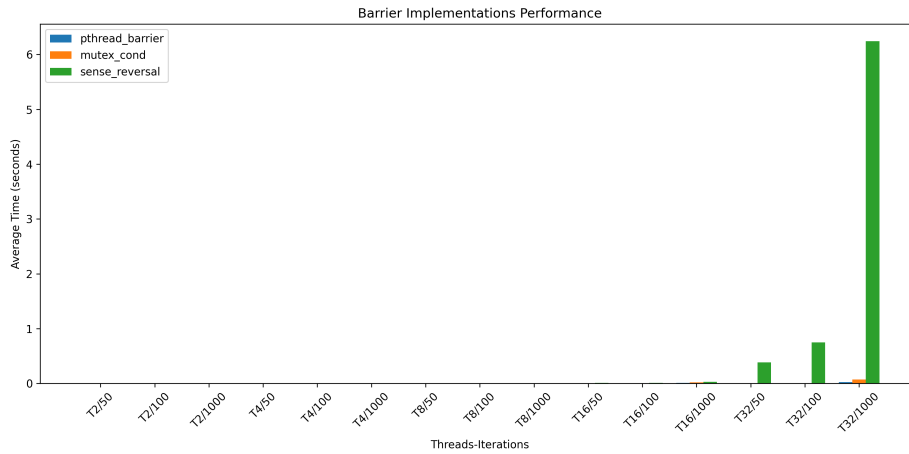
Στο δεύτερο πρόγραμμα υλοποιούμε το δικό μας φράγμα χρησιμοποιώντας *mutex* και *condition variable* για να αποφυγούμε την αναμονή σε εγρηγορήση και να μπορούμε να χρησιμοποιήσουμε αυτόν τον μηχανισμό για όλες της επαναληψεις. Ορίζουμε ένα *struct* όπου θα περιέχει έναν μετρητή νημάτων που έχουν φτάσει στο φράγμα (*barrier_count*), τον συνολικό αριθμό νημάτων *num_threads* τον συνολικό αριθμό επαναληψεων *n* τα *mutex*, *condition variable* και μια μεταβλητή *cycle* για να ξεχωρίσουν τα νήματα αν η αφύπνιση αφορά τον τρεχόντα γύρο ή κάποιον επομένο. Αν δεν υπήρχε ο *cycle* κάποια νήματα θα συνέχιζαν πιο νωρίς και κάποια θα περιμεναν ακόμα. Όταν κάθε νήμα φτάνει στο φράγμα αρχικά μπλοκαρουμε το *mutex* για να μπορούμε να αυξήσουμε τον μετρητή *barrier_count*. Αν το νήμα είναι το τελευταίο νήμα που φτάνει τότε επαναφέρουμε τον μετρητή στο μηδέν αυξάνουμε την μεταβλητή *cycle* για να ενημερώσουμε ότι ο κύκλος ολοκληρώθηκε και ξυπνάμε όλα τα υπολοίπα νήματα. Αντιθέτως αν κάποια νήματα έχουν φτάσει πιο νωρίς όσο το *cycle* παραμένει ίδιο θα μπλοκαρονται με `pthread_cond_wait` μέχρι να ειδοποιηθούν. Τέλος ελευθερώνουμε από την μνήμη ότι χώρο είχαμε δεσμεύσει για την υλοποίηση του προγράμματος.

Στο τρίτο πρόγραμμα υλοποιούμε το (*sense – reversal centralized barrier*) μια τεχνική που βασίζεται σε *busy waiting* αλλά παραμένει επαναχρησιμοποιήσιμη. Ορίζουμε ένα *struct* όπου θα περιέχει πληροφορίες για το *barrier*, δηλαδή έναν μετρητή νημάτων που έχουν φτάσει στο φράγμα (*barrier_count*), τον συνολικό αριθμό νημάτων *num_threads* το *mutex* και μια μεταβλητή *sense* που εναλλάσσεται μεταξύ 0 και 1 για να ξεχωρίσουμε τους γύρους. Η βασική ιδέα είναι ότι κάθε νήμα διατηρεί μια τοπική μεταβλητή *local_sense* η οποία αντιστρέφεται σε κάθε επαναληψη για να μπορούμε να ξεχωρίσουμε ποτέ βρισκόμαστε σε έναν νέο γύρο. Όταν ένα νήμα φτάσει στο *barrier* αυξάνουμε τον κοινό μετρητή μέσα στην κρίσιμη περιοχή προστατευμένη με το *mutex*. Αν είμαστε στο τελευταίο νήμα τότε επαναφέρουμε τα δεδομένα και ενημερώνουμε το *sense* ώστε να ταιριάζει με το *local_sense* για να σηματοδοτήσουμε ότι τα νήματα μπορούν να συνεχίσουν. Τα υπολοίπα νήματα που δεν ήταν τελευταία περιμένουν σε κατάσταση *busy – waiting* μέχρι να φτάσει και το τελευταίο νήμα. Τέλος ελευθερώνουμε από την μνήμη ότι χώρο είχαμε δεσμεύσει για την υλοποίηση του προγράμματος.

5.3 Πειραματικά αποτελέσματα και σχολιασμός

Threads-iterations	pthread_barrier	mutex_cond	sense_reversal
T2/50	0.000317	0.000289	0.000101
T2/100	0.000566	0.000520	0.000122
T2/1000	0.004054	0.004277	0.000428
T4/50	0.000418	0.000462	0.000205
T4/100	0.000654	0.000779	0.000294
T4/1000	0.005002	0.005408	0.001454
T8/50	0.000577	0.000663	0.000551
T8/100	0.000917	0.000974	0.002366
T8/1000	0.006000	0.007203	0.006707
T16/50	0.001251	0.001722	0.010696
T16/100	0.001842	0.003091	0.011437
T16/1000	0.011056	0.020517	0.031613
T32/50	0.002204	0.004926	0.384856
T32/100	0.003411	0.008363	0.749277
T32/1000	0.023623	0.071505	6.243802

(α') Barrier execution time table



(β') Barrier execution time

Σχήμα 8: Execution time.

Ο πίνακας με τα αποτελέσματα δείχνει ξεκαθάρια τις διαφορές στην επίδοση μεταξύ των 3 προγραμμάτων. Για χαμηλό αριθμό νημάτων (π.χ.:2) το *sense_reversal* έχουμε πολύ χαμηλούς χρόνους ξεπερνώντας τις άλλες δύο υλοποιήσεις. Αυτό γίνεται διότι η αναμονή στο *busy – waiting* αποδευγει το κόστος μπλοκαρίσματος. Ωστόσο καθώς αυξάνεται ο αριθμός των νημάτων ο *busy – waiting* δημιουργείται έντονη συμφορηση, δηλαδή πολλά νήματα σπινάρουν ταυτόχρονα και καταλάνωνουν *cpu*. Επομένως για αυτό τον λόγο η *sense_reversal* όταν αυξάνεται ο αριθμός των νημάτων ο χρόνος φτάνει πολλαπλασιος από τους άλλους δύο φραγμούς.

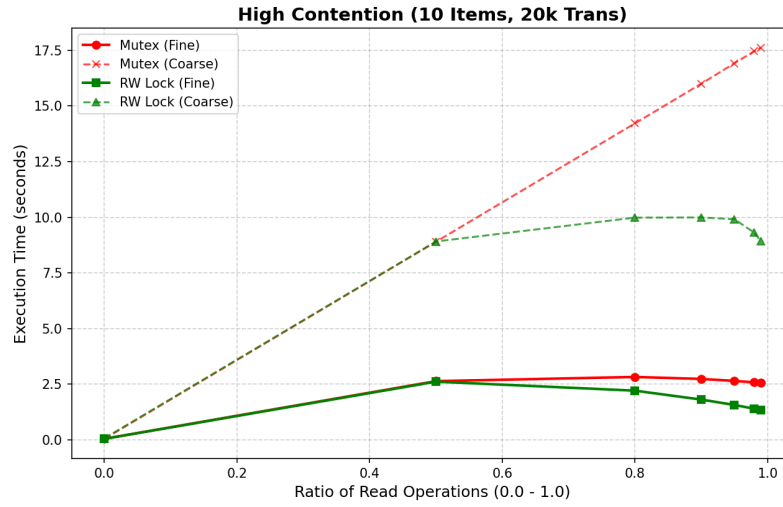
Στον υπολογιστή που υλοποιήσαμε την άσκηση παρέχονται 8 *cores* και 16 *logical cores* (*hardware threads*). Όταν ο αριθμός των νημάτων ξεπερνάει τον αριθμό των πυρήνων παρατηρούμε από το σχεδιαγράμμα έντονη επιδείνωση στην επίδοση και των 3 προγραμμάτων (κυρίως του *sense_reversal*), δηλαδή πολύ πιο υψηλούς χρόνους συγκριτικά με τις υλοποιήσεις των προγραμμάτων με χαμηλότερα *threads*. Αυτό συμβαίνει διότι τα νήματα που περιμένουν ενεργά καταλάνωνουν *CPU* χρόνο και το λειτουργικό σύστημα αναγκάζεται να μοιράσει τους πόρους μέσω χρονοπρογραμματισμού και *context switching*. Επομένως όταν $number_threads \leq cores$ όλα τα προγράμματα λειτουργούν σχεδόν αποδοτικά. Αντιθέτα όταν $number_threads > cores$ οι υλοποιήσεις με *pthread_barrier*, *mutex_cond* καθυστερούν λόγω *scheduling* ενώ οι υλοποιήσεις με *busy – waiting* (*sense – reversal*) υποβαθμίζονται δραματικά επειδή τα νήματα καταλάνωνουν *cpu* χωρίς να προσδεδουν.

Παρότι το *sense – reversal* βασίζεται σε *busy – waiting* παραμένει επαναχρησιμοποιησιμο χάρη στην τεχνική εναλλαγής της μεταβλητής *sense*. Όπως περιγράψαμε και παραπάνω κάθε νήμα θα περιέχει μια μεταβλητή που την ονομάζουμε *local_sense* η οποία αντιστρέφεται σε κάθε επαναλήψη. Αν είμαστε στο τελευταίο νήμα που φτάνει στο φράγμα ενημερώνει το *sense* της *struct* για να επιτρέψει στα υπόλοιπα να συνεχίσουν, διαφορετικά γίνεται *busy waiting* μέχρι να φτάσουν όλα τα νήματα στο φράγμα. Αυτή η

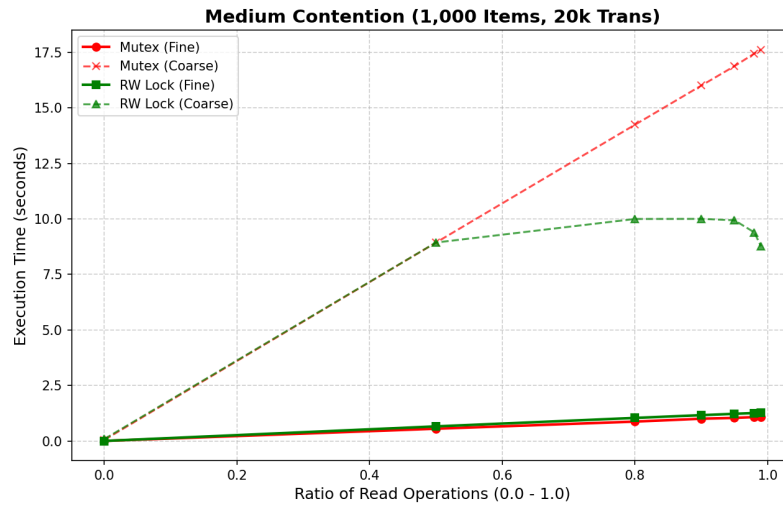
εναλλαγή του *sense* από 0 σε 1 και το αντιστρόφο εξασφαλίζει την επαναχρησιμοποίηση χωρίς να χρειαστεί να ξανααρχικοποιήσουμε τις μεταβλητές μας .

Συμπερασματικά το *pthread_barrier* που μας παραχεται από τη βιβλιοθήκη *Pthreads* είναι σταθερά αποδοτικό και απλό για τον χρήστη , το *mutex_cond* είναι ευελικό και σχετικά αποδοτικό σε περιβάλλον με υψηλό φορτίο, ενώ το *sense_reversal* είναι κατάλληλο για μικρό αριθμό *threads* αλλά δεν είναι καθόλου αποδοτικό όταν αυξάνεται ο αριθμός των νημάτων.

6 Όλα τα γραφήματα της άσκησης 4

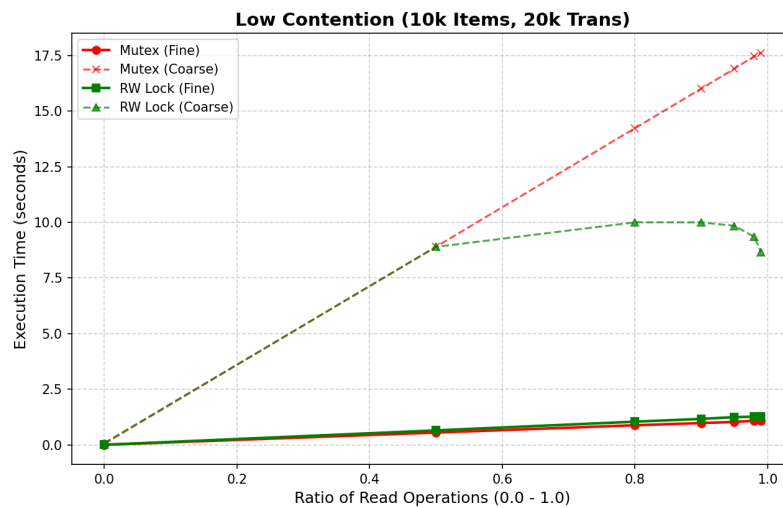


(α') High contestation with sleep



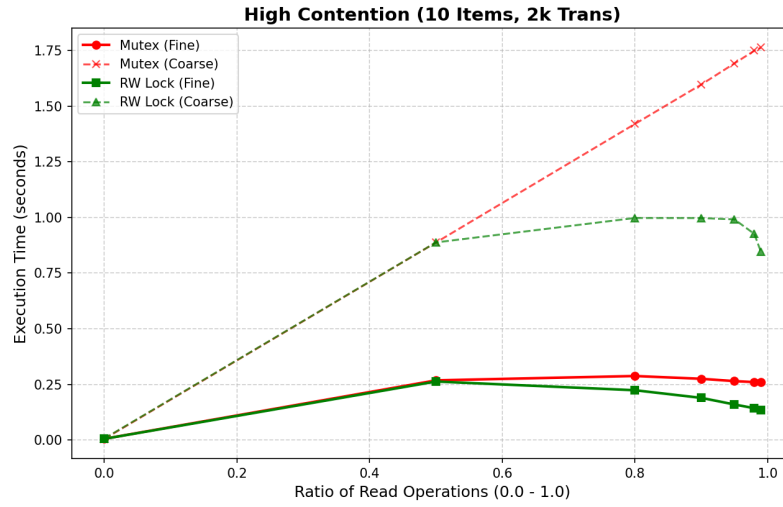
(β') Medium Contestation with sleep

$\Sigma\chi\eta\mu\alpha$ 9: Scenario time graphs with sleep.

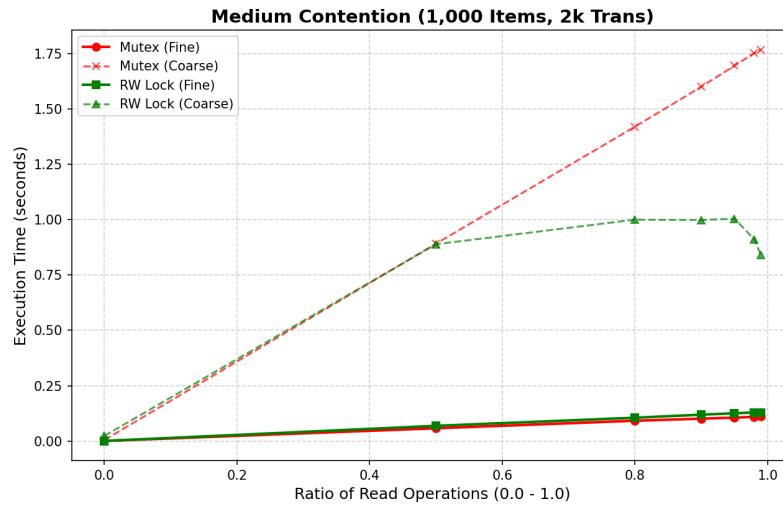


(α') Low Contention with sleep

$\Sigma\chi\eta\mu\alpha$ 10: Scenario time graphs with sleep.

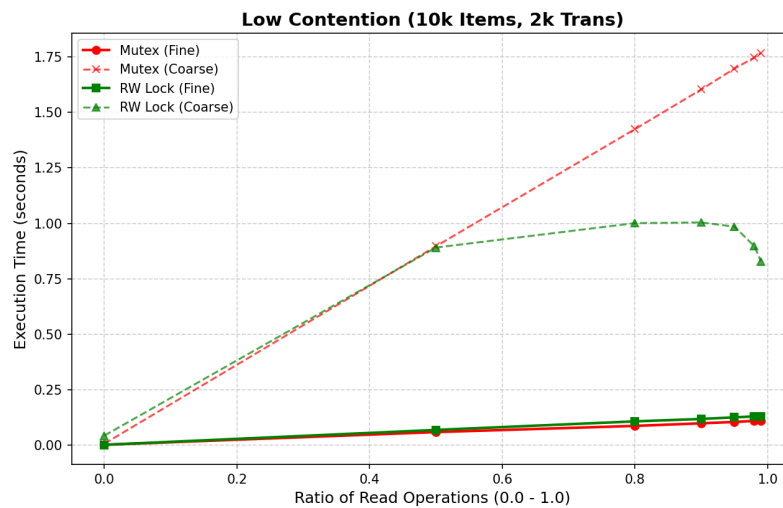


(α') High contestation with sleep



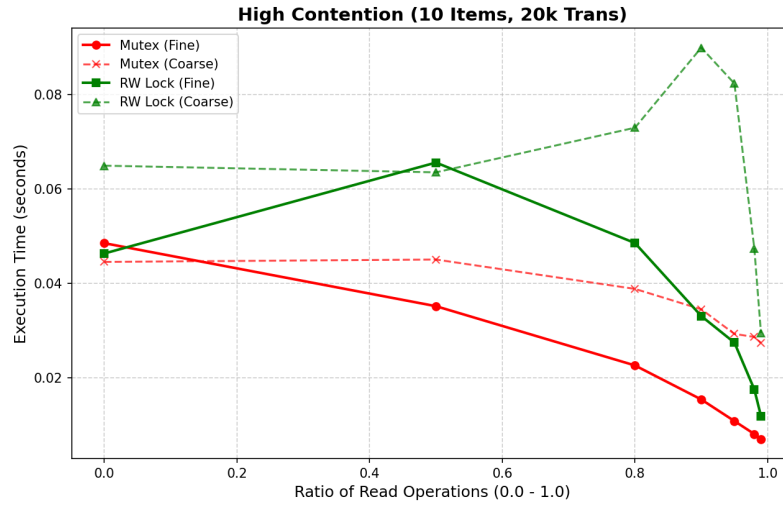
(β') Medium Contestation with sleep

$\Sigma\chi\eta\mu\alpha$ 11: Scenario time graphs with sleep.

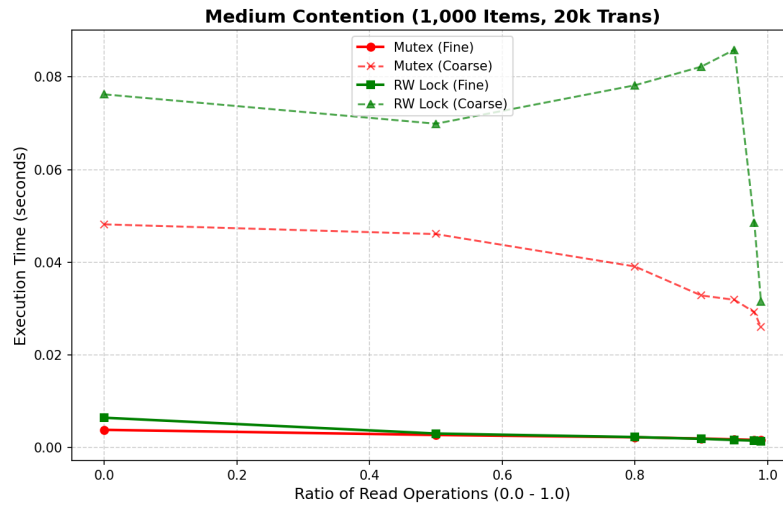


(α') Low Contention with sleep

$\Sigma\chi\eta\mu\alpha$ 12: Scenario time graphs with sleep and low transactions.

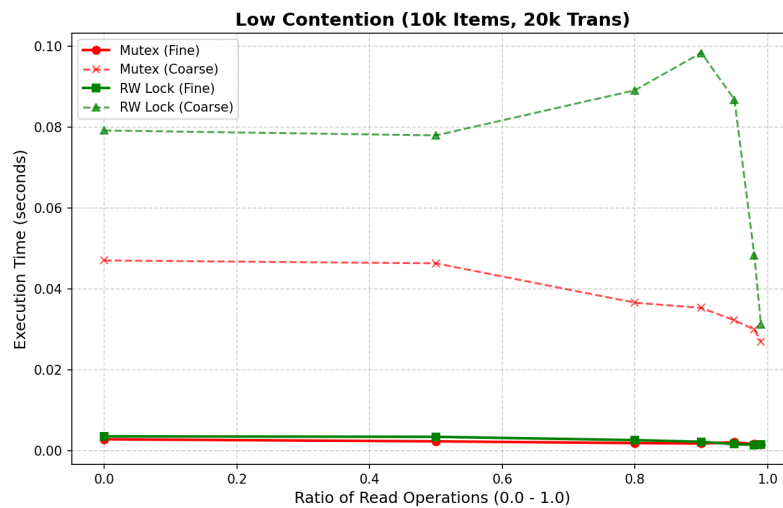


(α') High contention



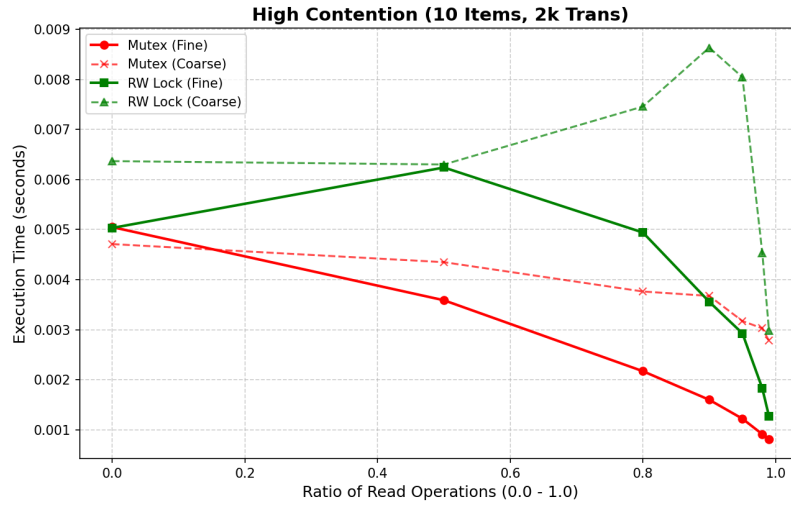
(β') Medium Contestion

Σχήμα 13: Scenario time graphs .

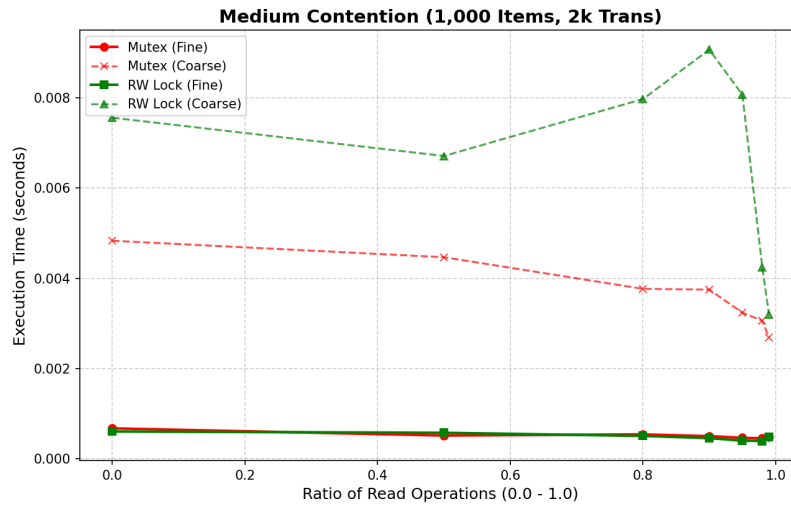


(α') Low Contestion

Σχήμα 14: Scenario time graphs with no sleep.

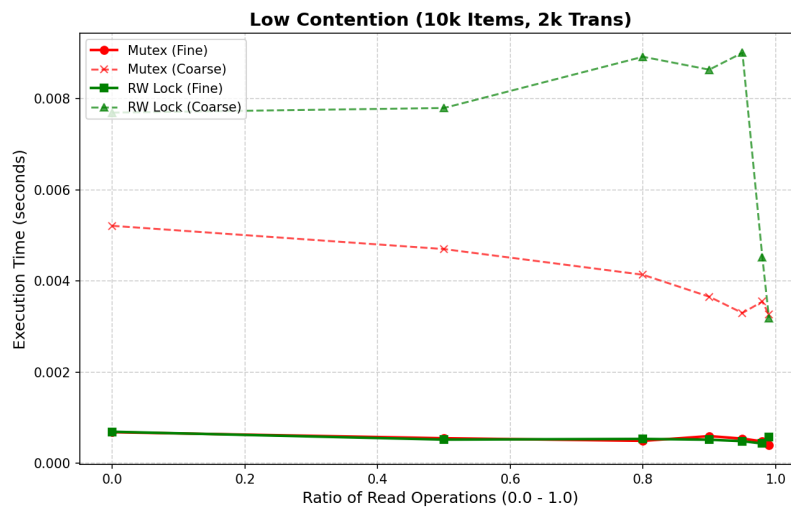


(α') High contention



(β') Medium Contestion

$\Sigma\chi\eta\mu\alpha$ 15: Scenario time graphs with sleep.



(α') Low Contestion

$\Sigma\chi\eta\mu\alpha$ 16: Scenario time graphs with sleep and low transactions.